课程笔记

消息传递：MPI 的语义与软同步

CMU 15-418/618 并行计算机架构与编程 · Lecture 17（Spring 2018）

lecture-to-notes

2026 年 10 月 29 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 14 分 27 秒
视频链接：`https://www.bilibili.com/video/BV1Qj411B7sZ/`

目录

1 为什么需要消息传递 **2**
1.1 与共享内存的分野 2
1.2 异步的本质与两种选择 3
1.3 本章小结 3

2 MPI 是什么：规范而非库 **4**
2.1 规范与实现 4
2.2 最小程序与错误处理 4
2.3 本章小结 5

3 运行环境与启动方式 **5**
3.1 为什么没有标准环境 5
3.2 本章小结 6

4 基本概念：通信器、类型与标签 **6**
4.1 通信器与 rank 6
4.2 数据类型 8
4.3 标签 8
4.4 本章小结 9

5 阻塞与非阻塞通信 **9**
5.1 发送接收的基本困难 9
5.2 阻塞通信 9
5.3 非阻塞通信 10
5.4 wait、test 与 probe 11
5.5 本章小结 12

6 集合操作与”MPI 其实很简单” **12**
6.1 六个函数覆盖九成场景 12
6.2 本章小结 14

7 软同步：同步发生在消息层 **14**
7.1 按通信而非按数据同步 14
7.2 本章小结 14

8 总结与延伸 **14**

1 为什么需要消息传递

这一讲安排得很紧凑：先用较快节奏过一遍 MPI，然后回到更多优化实例。讲者说 MPI 本身相当直白，所以没必要在这些事情上花太多时间。

1.1 与共享内存的分野

MPI 是消息传递接口，设计初衷是给超算提供一个”靠通信解决问题”的接口。

讲者对比得很清楚：在共享内存中，我们假设处理器可以通过共享内存通信，无论访问是否一致。但如果涉及不同机器，或者 NUMA 远端访问代价太高，我们就不会忽略这一点，而是常常改用消息传递——发一个消息、做大量工作、把结果交回来。

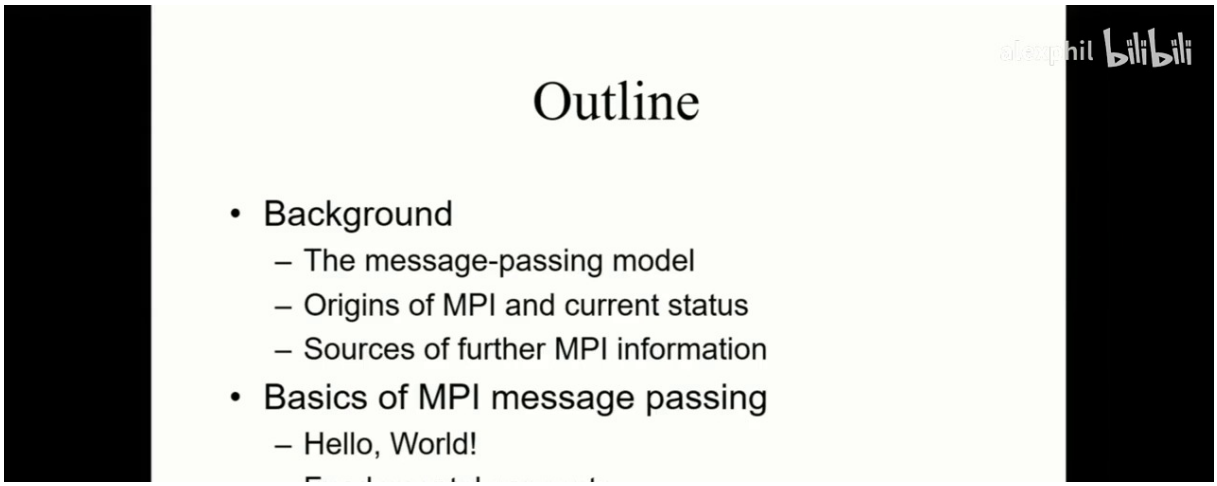


图 1: 本讲提纲：消息传递模型、MPI 的起源与现状，以及优化实例[1]

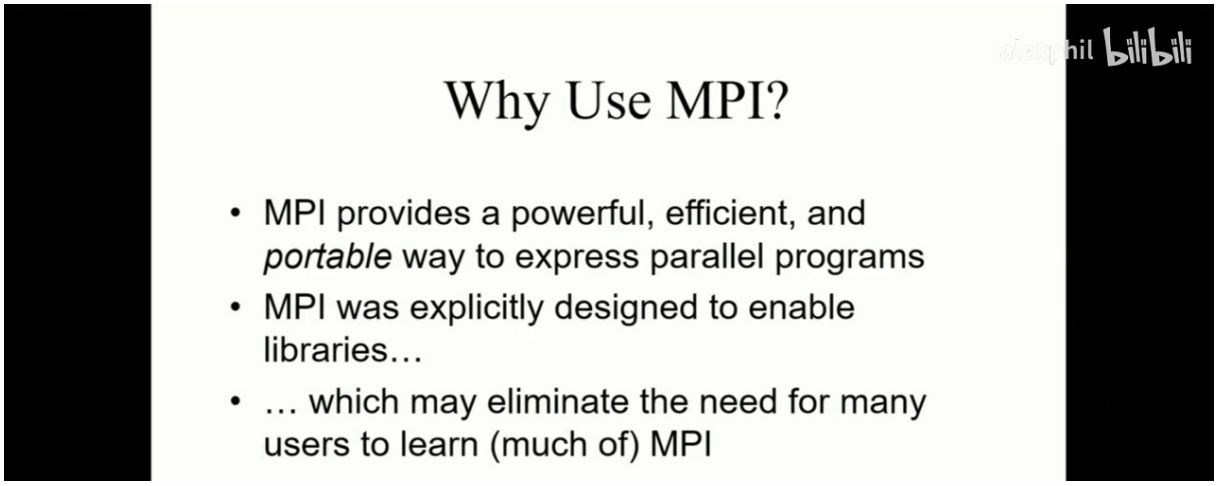


图 2: 为什么用 MPI：一种强大、高效、可移植的并行表达方式[2]

[1] 视频画面时间区间：00:00:30–00:00:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:03:00–00:03:15。若为裁剪图，时间区间仍对应原视频画面。

1.2 异步的本质与两种选择

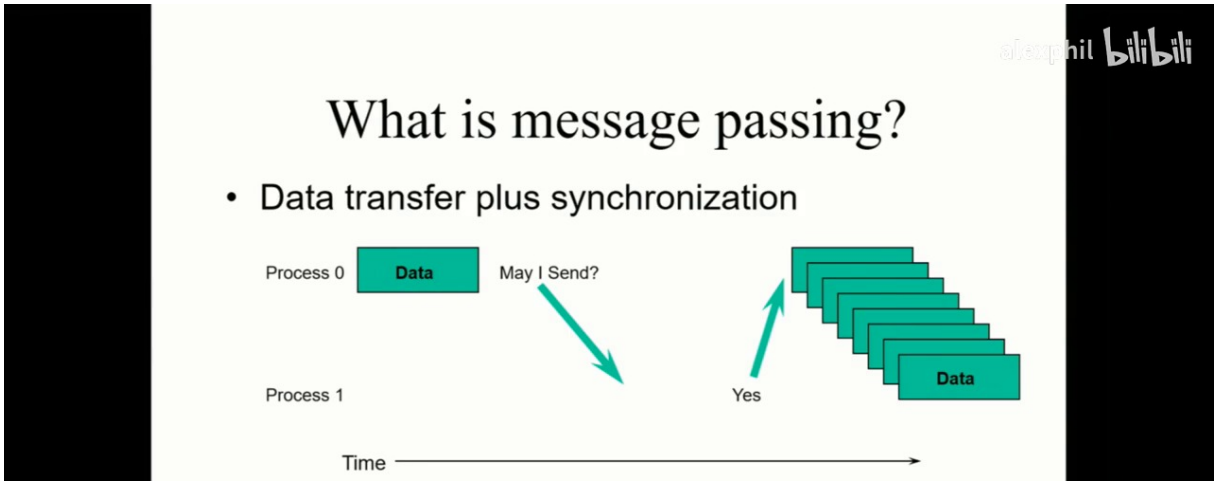


图 3: 什么是消息传递：数据传输加同步[3]

讲者强调消息传递本质上是异步的：发送方发消息给接收方，接收方把消息排队。然后我们有一个选择。

选项 A：作为发送方，我们等待接收方收到、处理、产生某种反应，并把回复发回来。

选项 B：发出即忘。我们把消息发出去，回去做其他正在做的事情，如果在意的话就周期性查看回复。

轮询的代价与频率是一对权衡

周期性检查回复比一直等待更灵活，但每次检查都要付一次系统调用或网络栈开销。检查太频繁会浪费 CPU，太稀疏则会错过及时响应。因此轮询间隔应与预期消息到达的粒度匹配。

他给了一个很贴切的类比：你可以寄一封信，然后每天站在信箱旁等回信；也可以寄出去，然后每天早上查一次邮箱，白天该干什么干什么。

阻塞与非阻塞不是风格差异而是结构差异

阻塞通信把”等待”变成调用者的状态，因此调用者在这段时间内无法做任何其他事。非阻塞通信把”等待”变成可查询的状态，调用者可以在等待期间计算。这个差别决定了程序能否重叠通信与计算，也就决定了通信延迟是否被隐藏。

1.3 本章小结

MPI 是为超算设计的消息传递接口，用于处理器无法靠共享内存廉价通信的场合。消息传递本质异步：发送方发出、接收方排队，之后可选择等待回复或发出即忘。这个选择对应阻塞与非阻塞两套语义，而它决定通信延迟能否被计算掩盖。

[3]视频画面时间区间：00:06:00–00:06:15。若为裁剪图，时间区间仍对应原视频画面。

2 MPI 是什么：规范而非库

2.1 规范与实现



图 4: 最小的 MPI 程序：只做初始化与关闭[4]

讲者明确一点：MPI 本身是一个接口，不是库。它是一份规范，由许多不同的库在不同系统上实现。

规范的抽象边界决定了可移植性的范围

MPI 规范只保证调用语义，不保证性能特征。因此同一份程序在不同实现上正确性一致，但延迟与带宽可能差数倍。跨平台调优时必须重新测量，不能沿用旧参数。

因此它规定通信是什么样子，但不规定你怎么实现。

2.2 最小程序与错误处理

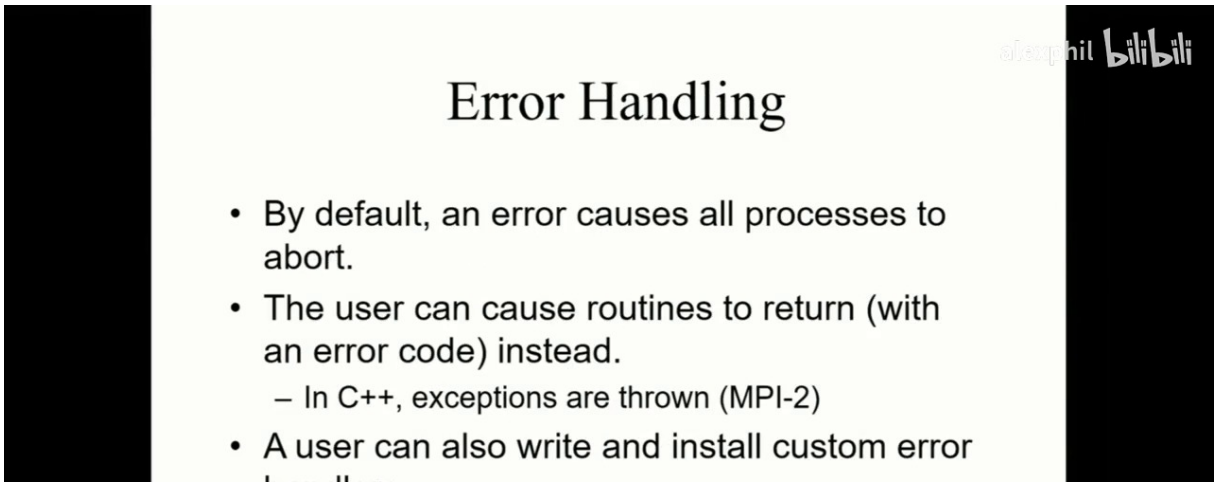


图 5: 错误处理：默认出错使所有进程终止，也可返回错误码[5]

[4]视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。

最小的 MPI 程序只做两件事：初始化 MPI 库，然后关闭。

讲者打趣说，不要指望这个程序带来显著性能提升。但他可以保证：它会给你和第一次跑 Hello World 一样多的即时满足感——你会说「好，MPI 能跑了」，然后呢?

默认终止语义对长时运行作业不友好

默认出错即全体终止，意味着一个进程的局部错误会毁掉整个作业。在需要长时间运行的场景里，通常改为返回错误码并由程序自行决定是否降级或重试，但这要求每个调用点都检查返回值。

关于错误处理：默认情况下出错会让所有进程终止。用户可以改为让例程返回错误码。讲者补充说，若你用 Fortran，通常返回错误码；若用 C++，会得到异常。

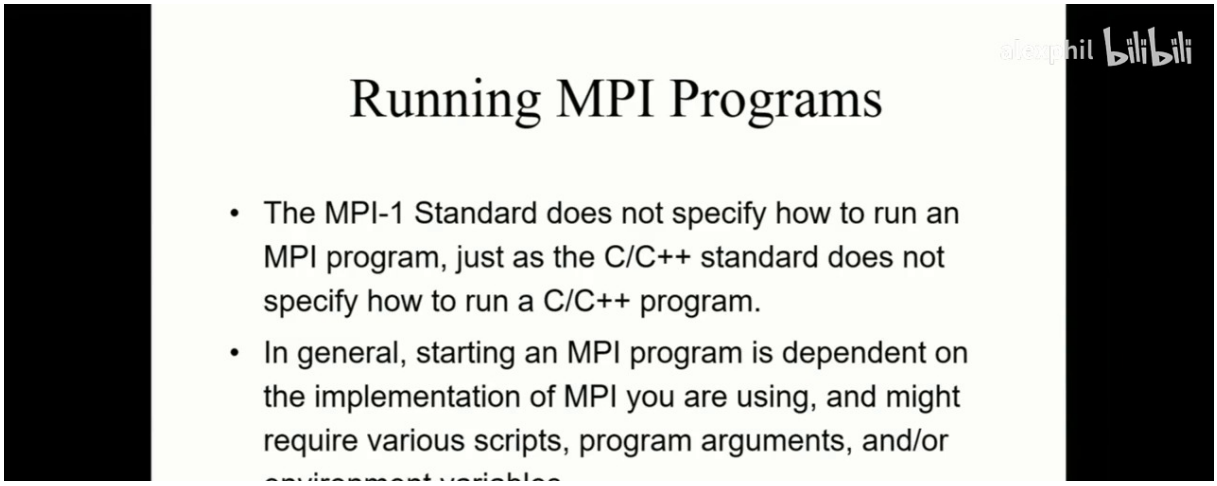


图 6: 运行 MPI 程序：标准不规定运行方式[6]

2.3 本章小结

MPI 是规范而非库，只规定通信语义不规定实现。最小程序只做初始化与关闭，它的价值是确认环境可用而非提升性能。错误处理可选返回码或异常，默认出错则全体终止。

规范之外还有一层：程序实际如何被启动与连接。

3 运行环境与启动方式

3.1 为什么没有标准环境

讲者指出一个容易被忽略的事实：运行 MPI 程序没有标准化的环境。这就像运行 C++ 程序也没有标准化环境一样——那不是接口或库的函数，而是运行时环境的函数。

他提到有建议说：若运行 MPI 程序，命令应叫 `mpiexec`，然后接程序名。这样会调用运行时，由它处理所有线程与已初始化的通信设施。

但这只是建议，并非所有 MPI 系统都必须这样。

[5] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

规范与运行时的分工

MPI 规范定义的是”调用语义”，而进程如何启动、如何发现彼此、如何建立连接属于运行时实现。因此同一份 MPI 程序在不同集群上可能需要不同的启动命令与参数。移植时出问题往往在运行时层，而不在 API 层。

3.2 本章小结

MPI 是规范而非库，只规定通信语义不规定实现。最小程序只做初始化与关闭；错误处理可选返回码或异常。运行 MPI 程序没有标准环境，`mpiexec` 只是建议。规范与运行时的分工解释了为什么移植问题常出在运行时层。

4 基本概念：通信器、类型与标签

4.1 通信器与 rank

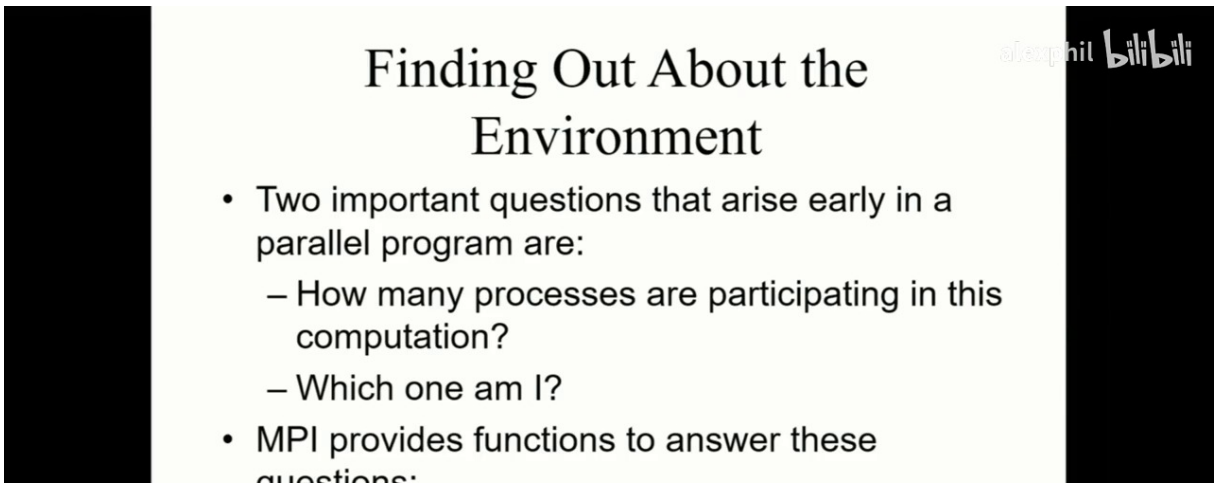


图 7: 了解运行环境：并行程序早期要问的两个问题[7]



图 8: 改进版 Hello：获取组内进程数与自身 rank[8]

[7]视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。

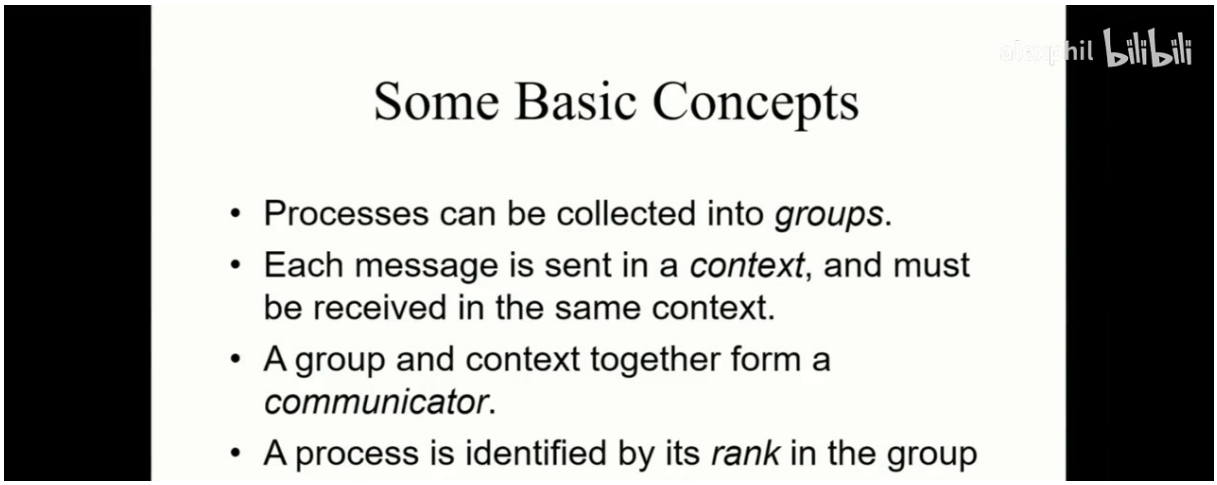


图 9: 基本概念：进程组成组，每条消息在通信器上下文中发送[9]

rank 是逻辑标识而非物理位置

rank 只表示进程在通信器内的编号，不携带拓扑信息。因此”rank 0 在第一个插槽” 这类假设不可移植。要获得拓扑信息，需要使用专门的拓扑查询接口或由启动器显式指定映射。

讲者说并行程序早期会问两个重要问题：有多少个进程？我是哪一个？

进程由它在通信器中的 rank 标识。有一个默认通信器，其组包含所有初始进程。这个默认通信器的名字是 `MPI_COMM_WORLD`。初始化 MPI 后你就属于它，除非你加入别的通信器。

通信器这个概念同时捕获了两件事：组成员关系，以及在组内通信的能力。讲者说这两件事名义上是分开的，但通信器把上下文与通信能力合并在一起。

通信器既是名字空间也是隔离域

同一个 rank 编号在不同通信器中指不同进程。因此通信器提供了消息隔离：在一个通信器里发的消息不会被另一个通信器的接收误取。这使得库可以安全地使用私有通信器，而不干扰调用者的消息。

[8] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

[9] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

4.2 数据类型

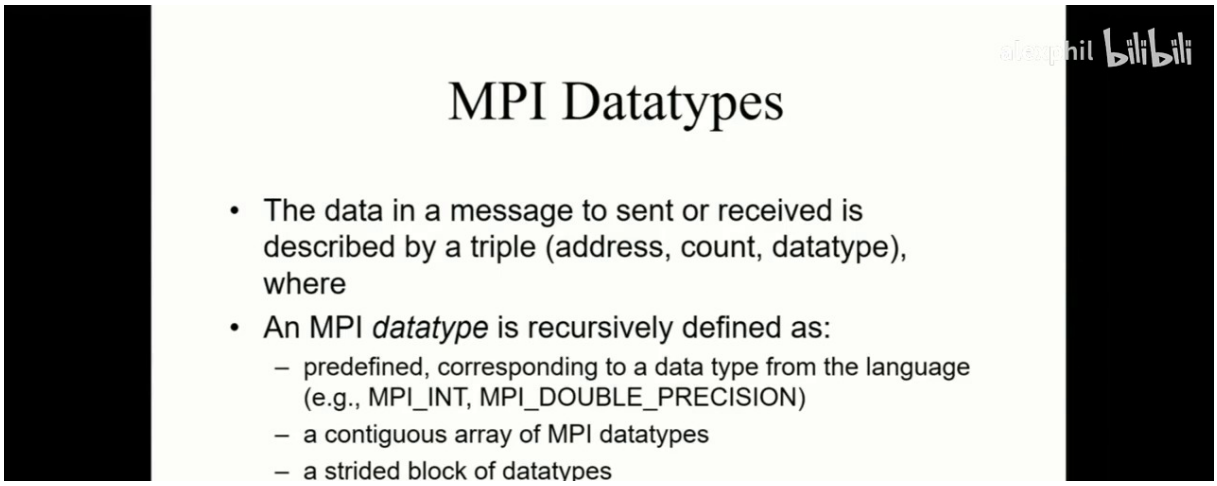


图 10: MPI 数据类型：由地址、计数与类型三元组描述消息数据[10]

因为 MPI 设计为在异构环境中工作，它有自己的一套数据类型：`MPI_INT`、`MPI_DOUBLE` 等等。

使用 MPI 数据类型带来两个好处：一是可以自由跨不同类型的系统工作，二是我们确切知道数据有多大。

类型标准化是异构通信的前提

若发送方与接收方的 int 宽度不同，直接按字节复制会得到错位的数据。MPI 的类型描述让运行库能在两侧之间做转换。因此通信时不应传裸类型的大小，而应传对应的 MPI 类型。

讲者点出一个 C 语言的经典问题：如果你说 `int`，人们会问这个 int 有多大。MPI 通过标准化数据类型、表示与大小解决了这个问题。

4.3 标签

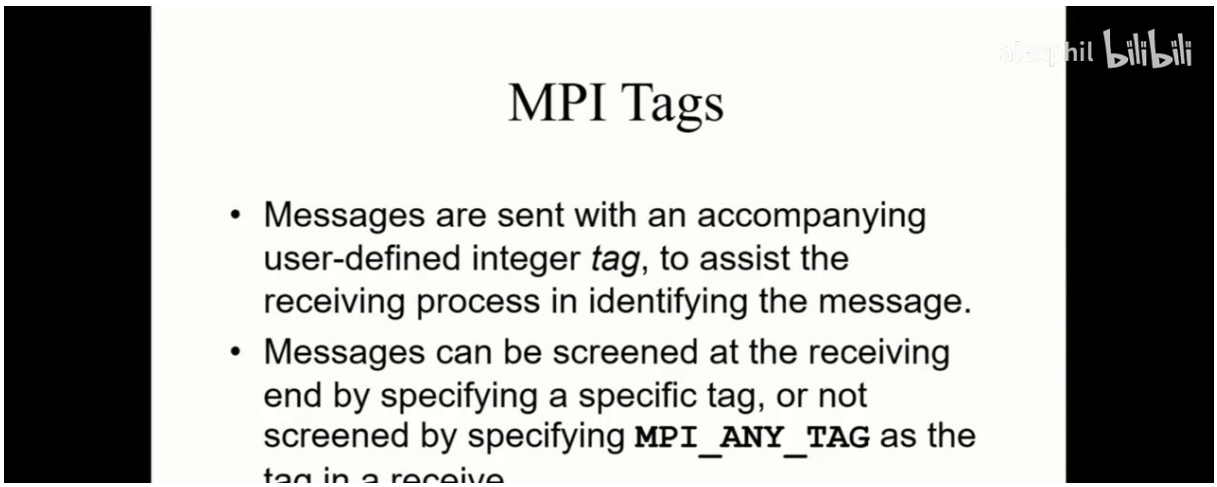


图 11: MPI 标签：用用户定义的整数标记消息类型[11]

[10] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

MPI 标签曾经被用得很多。有人把它叫消息类型：发消息时可以打上标记说明这是哪类消息，接收方据此决定如何处理，从而实现对消息类型的内部组织。

讲者说标签现在仍然可以用。

4.4 本章小结

通信器同时捕获组成员关系与通信上下文，进程由 rank 标识，默认通信器含全部初始进程。MPI 自带数据类型以支持异构环境并确知大小。标签用于标记消息类型，接收方据此决定处理方式。

5 阻塞与非阻塞通信

5.1 发送接收的基本困难

讲者先描述经典问题：发送方想发数据，接收方可能还没准备好；接收方在等东西，发送方可能还没发。存在突发流量时发送方发了很多而接收方尚未就绪；也有接收方闲着而发送方还没准备好的时候。

缓冲就在中间起作用：暂存这些突发，在接收方就绪时把它们排空。

所需缓冲深度与发送速率和接收速率的差值有关：

$$B_{\min} \approx (r_{\text{send}} - r_{\text{recv}}) \cdot \Delta t_{\text{burst}}$$

- r_{send}：发送方突发期间的发送速率；
- r_{recv}：接收方在同期能消费的速率；
- Δt_{burst}：突发持续时间；
- 差值乘以时长即为需要暂存的字节数。

若缓冲不足，发送方就被迫阻塞，非阻塞语义随之退化。

5.2 阻塞通信

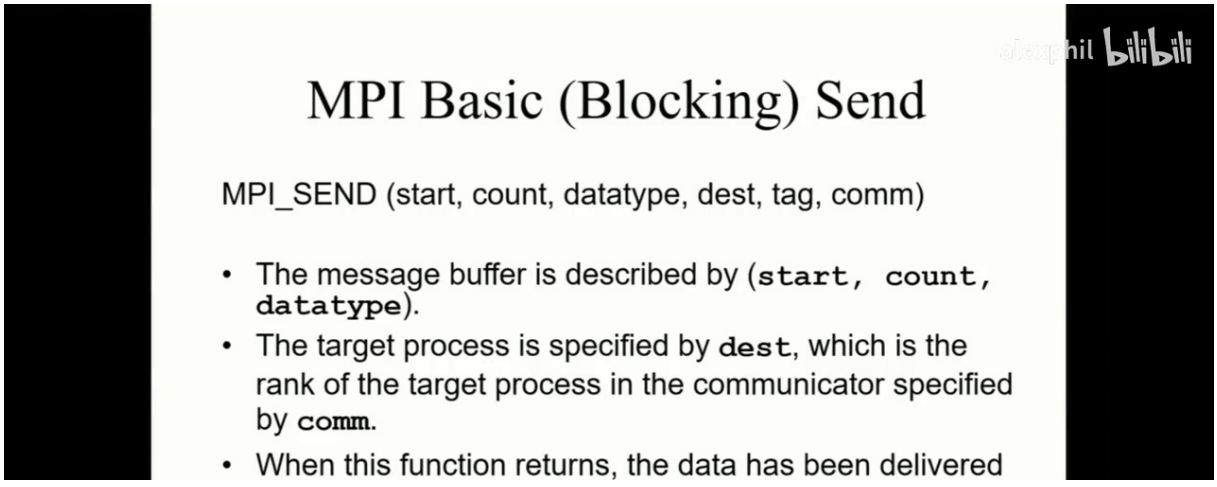


图 12: MPI 基本（阻塞）发送：`MPI_Send` 的参数[12]

[11] 视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

讲者说阻塞通信很简单：发出、等确认、然后你知道宇宙的状态并继续。

问题在于它把本可并行的事情串行化了。有些东西本可以继续推进，却被冻结在那里等别人行动。

阻塞调用的代价可以用等待时间占循环总时间的比例来衡量：

$$f_{\text{stall}} = \frac{T_{\text{wait}}}{T_{\text{wait}} + T_{\text{compute}}}$$

- T_{wait}：每次阻塞调用中实际等待对端的时长；
- T_{compute}：同一轮中可与之重叠的计算时长；
- 若 $T_{\text{wait}} \gg T_{\text{compute}}$，则绝大部分时间被冻结；
- 非阻塞形式的目标就是让 T_{compute} 覆盖 T_{wait}，使 $f_{\text{stall}} \to 0$。

阻塞通信的代价在等待期间被放大

一次阻塞发送可能只等几微秒，但若在循环中反复调用，累计等待会吃掉大量本可用于计算的时间。因此在高性能代码中，阻塞调用常被移出热循环，或改用非阻塞形式批量提交、最后统一等待。

5.3 非阻塞通信

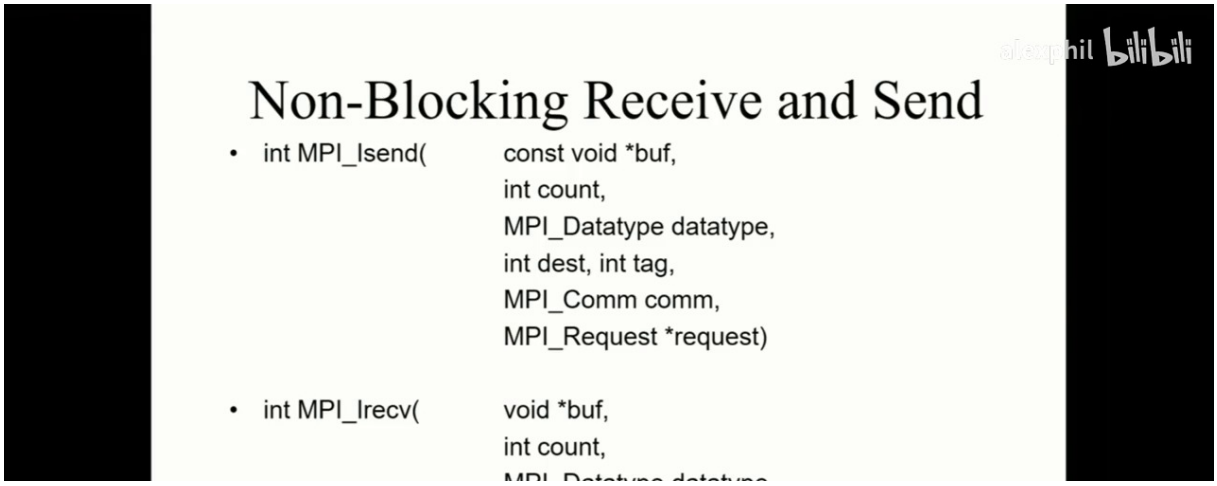


图 13：非阻塞接收与发送：`MPI_Isend` 等接口[13]

非阻塞发送与接收放宽了这个模型。

非阻塞发送：你发出去，它被排队，然后你继续干自己的事。

非阻塞接收：你请求某个东西，要么拿到、要么还没有，但你不阻塞。

于是我们可以这样做：查看东西是否到了，然后回去做别的工作；再查看，再做别的工作。这就允许我们把计算重叠到等待时间里。

[12] 视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

5.4 wait、test 与 probe

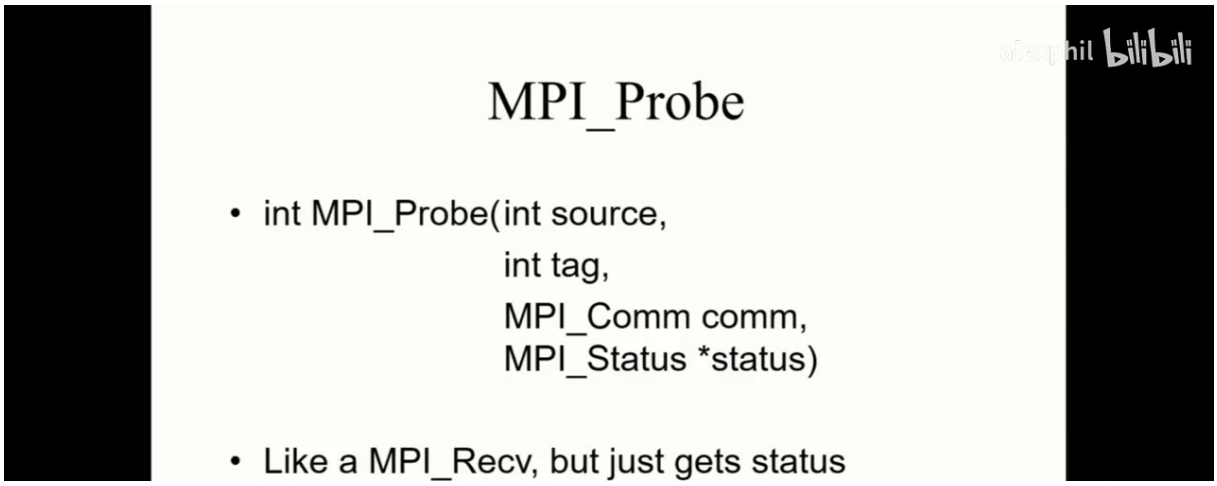


图 14: `MPI_Probe`：只取状态，不取数据[14]

讲者说明处理非阻塞操作的两个手段。`MPI_Wait` 与 `MPI_Test`：若我们有一个非阻塞操作，而现在想等它——因为能做的别的事都做完了——就调用 wait，此时会阻塞直到完成。

若用 test 并拿到状态，它可以告诉我们比如这次通信是否失败。

Probe 解决”不知道要收多少”的问题

接收方常需要先知道消息长度才能分配缓冲区。Probe 只读取消息头（来源、标签、大小），不消费消息体，因此可以先分配再接收。这是消息传递中处理变长消息的标准手法。

`MPI_Probe` 像一个接收，但只取状态、不取数据。讲者说它像是”偷看一眼”。

$$\text{进度} = \begin{cases} \text{MPI_Test\&非阻塞，立即返回是否完成} \\ \text{MPI_Wait\&阻塞直到完成} \\ \text{MPI_Probe\&只看有没有消息，不取走} \end{cases}$$

- `MPI_Test`：适合在计算之间周期性检查，不打断计算流；
- `MPI_Wait`：适合已无事可做时，把剩余等待交给系统；
- `MPI_Probe`：适合先得知消息大小与来源，再决定分配多少缓冲区。

非阻塞通信有一个典型的配套用法，值得单独说明：先把若干发送与接收全部以非阻塞形式提交，在中间插入计算，最后统一调用 wait 等待全部完成。这样一次等待就覆盖了多个通信，而不是逐个阻塞。代价是需要为每个未完成操作保存一个请求句柄，且缓冲区在所有操作完成前不能复用。这个模式在需要交换边界数据的结构化网格应用中很常见：先发起所有邻居的发送与接收，再计算内部区域，最后等待边界数据到达。它把通信延迟完全藏在了内部计算之后。

[14] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

5.5 本章小结

发送方与接收方的就绪时间不匹配，缓冲用于吸收突发。阻塞通信简单但把可并行的部分串行化。非阻塞发送入队即返回、非阻塞接收不阻塞，因此可把等待时间用于计算。`MPI_Wait` 阻塞等待完成、`MPI_Test` 非阻塞查询、`MPI_Probe` 只取状态不取数据。

6 集合操作与"MPI 其实很简单"

6.1 六个函数覆盖九成场景

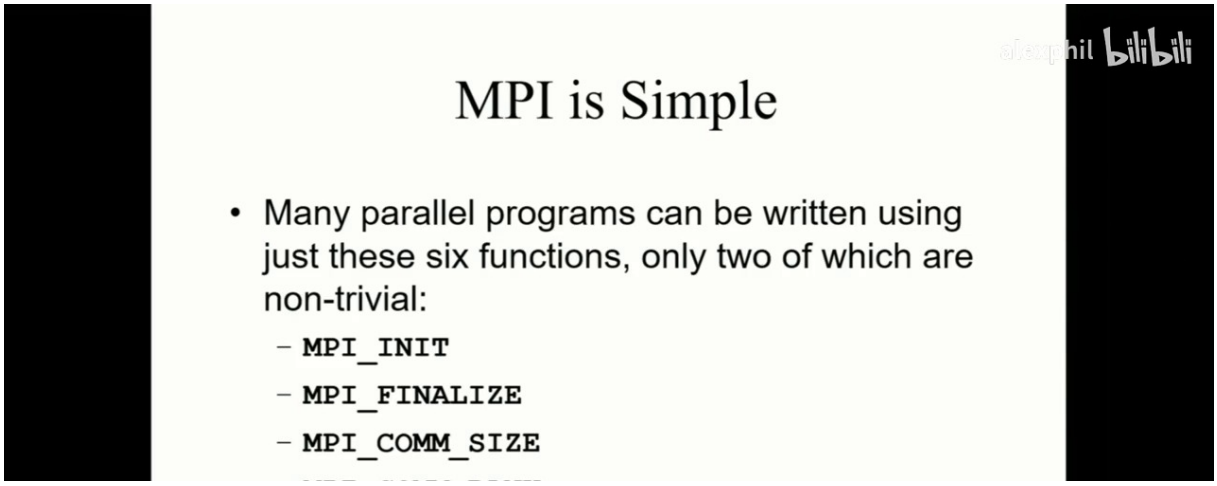


图 15: MPI 其实很简单：六个基本函数足以写多数并行程序[15]

讲者给出一个很有说服力的判断：许多并行程序只用这六个函数就能写出来，它们大概覆盖了我们会用到的 90%。

若再加上两个非阻塞操作与 probe，大约九个函数。他说这就是 MPI 的全部。

API 简单不等于并行化简单

六个函数足以表达多数通信模式，但"如何把问题切成可通信的块、如何安排通信与计算的重叠、如何避免某个进程成为瓶颈"这些问题无法由 API 解决。因此学习 MPI 的成本主要不在语法，而在分解策略。

而 MPI 的诀窍不在于怎么用这些库，而在于怎么解决问题。

[15] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

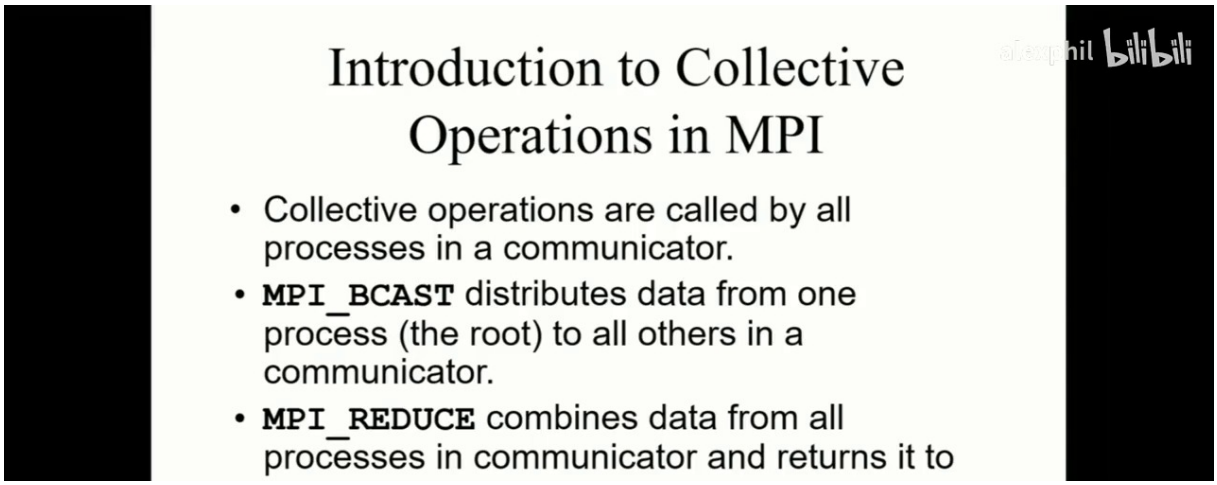


图 16: 集合操作入门：由所有进程共同调用[16]

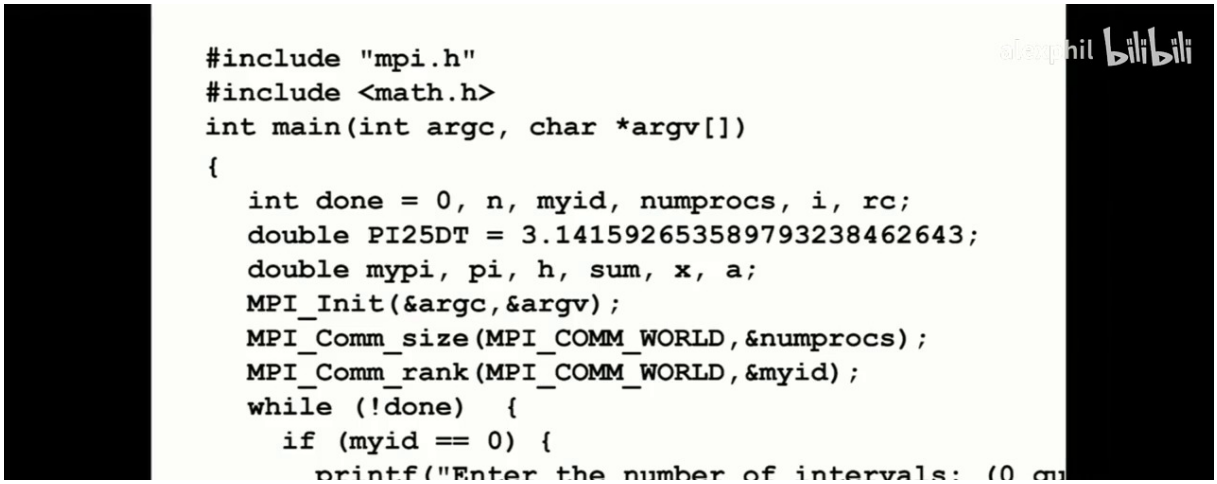


图 17: 示例：用 C 语言算圆周率，主从结构[17]

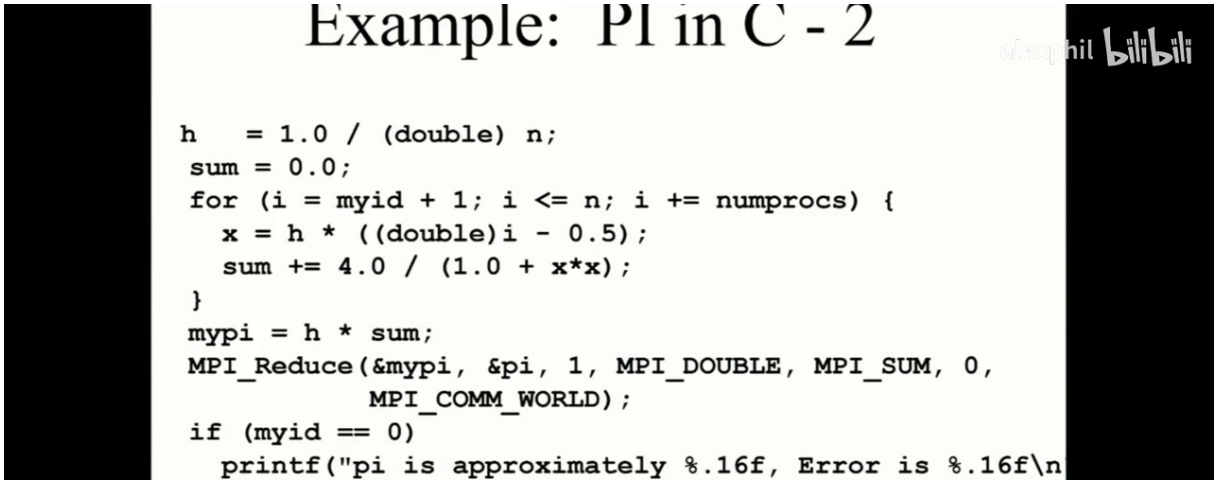


图 18: 圆周率示例续：各进程分片累加再归约[18]

[16]视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

[17]视频画面时间区间：00:15:00–00:15:15。若为裁剪图，时间区间仍对应原视频画面。

软同步还有一个实际好处：它天然适合流水线式的分解。若把问题切成若干阶段，让不同进程分别处理不同阶段的数据块，那么每个进程只需在把结果交给下游时同步一次。上游不必知道下游内部如何实现，下游也不必关心数据从哪来。这种松耦合让系统更容易扩展——增加一级处理只需在链上插入一个新进程并修改相邻两级的通信端点，而不必改动整个程序的结构。共享内存方案要做到同样的扩展性，通常需要重新设计锁与数据布局。

6.2 本章小结

六个基本函数覆盖约九成使用场景，加上非阻塞与 probe 共约九个。集合操作由所有进程共同调用，用于广播、归约与扫描等集体模式。圆周率示例展示了典型的主从结构：各进程分片累加，再归约汇总。而 MPI 的诀窍不在 API 用法，而在问题分解。

最后一个概念值得单独讨论：消息传递改变了同步发生的位置。

7 软同步：同步发生在消息层

7.1 按通信而非按数据同步

讲者最后给出一个概念性的判断，值得单独记录。

消息传递是一种很好的解决问题的方式，因为它允许一种”软同步”：一个进程可以做一堆工作，做完后发一条消息。

于是你是在通信上同步，而不是在数据上同步。你不共享内存，因此不会为了抢同一个东西而堆积。你取到某个东西、处理它、把结果发出去。

软同步把”等待”从数据层移到消息层

共享内存的同步发生在数据上：多个线程抢同一块内存，必须用锁或原子操作协调。消息传递的同步发生在消息上：进程各自持有私有数据，只在发送与接收时对齐。因此消息传递天然避免了数据竞争，代价是必须显式搬运数据。

7.2 本章小结

六个基本函数覆盖约九成使用场景，加上非阻塞与 probe 共约九个；难点不在 API 而在问题分解。消息传递提供软同步：干完活再发消息，按通信同步而非按数据同步，因此不共享内存、不会为同一目标堆积。

8 总结与延伸

讲者在结尾处理了一些课程事务与反馈，随后结束。

把全讲串起来，这是一份关于”当处理器之间不能共享内存时如何表达并行”的速览。MPI 是为超算设计的消息传递接口，用于共享内存代价过高或跨机器的场合；它的模式是发消息、做大量工作、交回结果。消息传递本质异步：发送方发出、接收方排队，之后可选择等待回复（阻塞）或发出即忘（非阻塞），后者允许把计算重叠到等待时间。MPI 是规范而非库，只规定通信语义不规定实现；运行 MPI 程序没有标准环境，`mpiexec` 只是建议。通信器同时捕获组成员关系与通信上下文，进程由 rank 标识，默认通信器含全部初始进程；MPI 自带数据类型以支持异构环境并确知

[18] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

大小；标签用于标记消息类型。发送方与接收方就绪时间不匹配，缓冲用于吸收突发。阻塞通信简单但串行化本可并行的部分；非阻塞发送入队即返回、接收不阻塞，配合 `MPI_Wait`、`MPI_Test` 与 `MPI_Probe` 可以灵活控制。最后，六个基本函数覆盖约九成场景，而消息传递提供软同步——按通信同步而非按数据同步。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 消息传递与共享内存的差别不只是" 能不能共享数据"，而是" 同步发生在哪一层"。共享内存同步数据，消息传递同步消息。这个区别决定了竞争与死锁的形态完全不同。
- MPI 是规范而非库，这一点有实际后果。同一份程序在不同集群上可能需要不同的启动方式与参数，移植问题常出在运行时层而非 API 层。
- 非阻塞不等于更快，而是更可控。它把等待变成可查询的状态，但需要程序自己安排" 什么时候检查"。若检查得太晚，收益会被浪费；若检查得太频繁，开销会抵消收益。
- 软同步的代价是显式数据搬运。消息传递避免了锁与数据竞争，但每次通信都要复制数据。因此它对" 计算量相对通信量大" 的问题最有利，这与第 23 讲片上与外接加速器的判据同构。

这堂课的边界在于它是速览：讲者明确说 MPI 本身很直白，因此只花很短时间过一遍，随后转入优化实例。因此本文不含完整的 MPI 调用参考与集合操作的细节语义。

两个可以直接从本讲数据提出的下一步问题：其一，非阻塞通信要求程序安排检查时机，那么检查频率应当如何与计算粒度匹配才不浪费收益？其二，软同步避免了数据竞争，但显式搬运数据会带来复制开销，在什么计算通信比之下消息传递会优于共享内存？